Oviyam

Notarizing module

User's guide

made in v19 R6

Version	Date	Writer	Comments
2.5.01	Monday, 3 May 2021	OG	first version
2.5.02	Wednesday, 5 May 2021	OG	Added a "4D like" Signing - based on an sh file.
2.5.03	Thursday, 6 May 2021	OG	Added in [Products] "app-specific password" field.
2.5.04	Friday, 7 May 2021	OG	Add a step in the process: staple.
2.5.05	Wednesday, 26 May 2021	OG	New process workflow picture for staple.
2.6.00	Monday, 26 December 2022	OG	Option to use the new "notarytool" command.

Protée
IT, app design

Protée sarl
31, rue Sainte-Marthe
31000 TOULOUSE

Tél : +33 (0)970 46 56 46
Mob : +33 (0)6 3718 5941
www.protee.org / info@protee.org

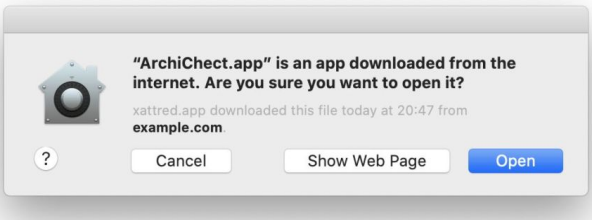
Introduction

Signing and Notarizing

Notarization gives users more confidence that the Developer ID-signed software you distribute has been checked by Apple for malicious components. Notarization is not App Review. The Apple notary service is an automated system that scans your software for malicious content, checks for code-signing issues, and returns the results to you quickly. If there are no issues, the notary service generates a ticket for you to staple to your software; the notary service also publishes that ticket online where Gatekeeper can find it.

When the user first installs or runs your software, the presence of a ticket (either online or attached to the executable) tells Gatekeeper that Apple notarized the software. Gatekeeper then places descriptive information in the initial launch dialog to help the user make an informed choice about whether to launch the app.

This is a part of GateKeeper, made to protect your computer, even if it is not funny for a developper to have the computer more and more controlled by an OS constructor!



Notarize Your App Automatically as Part of the Distribution Process

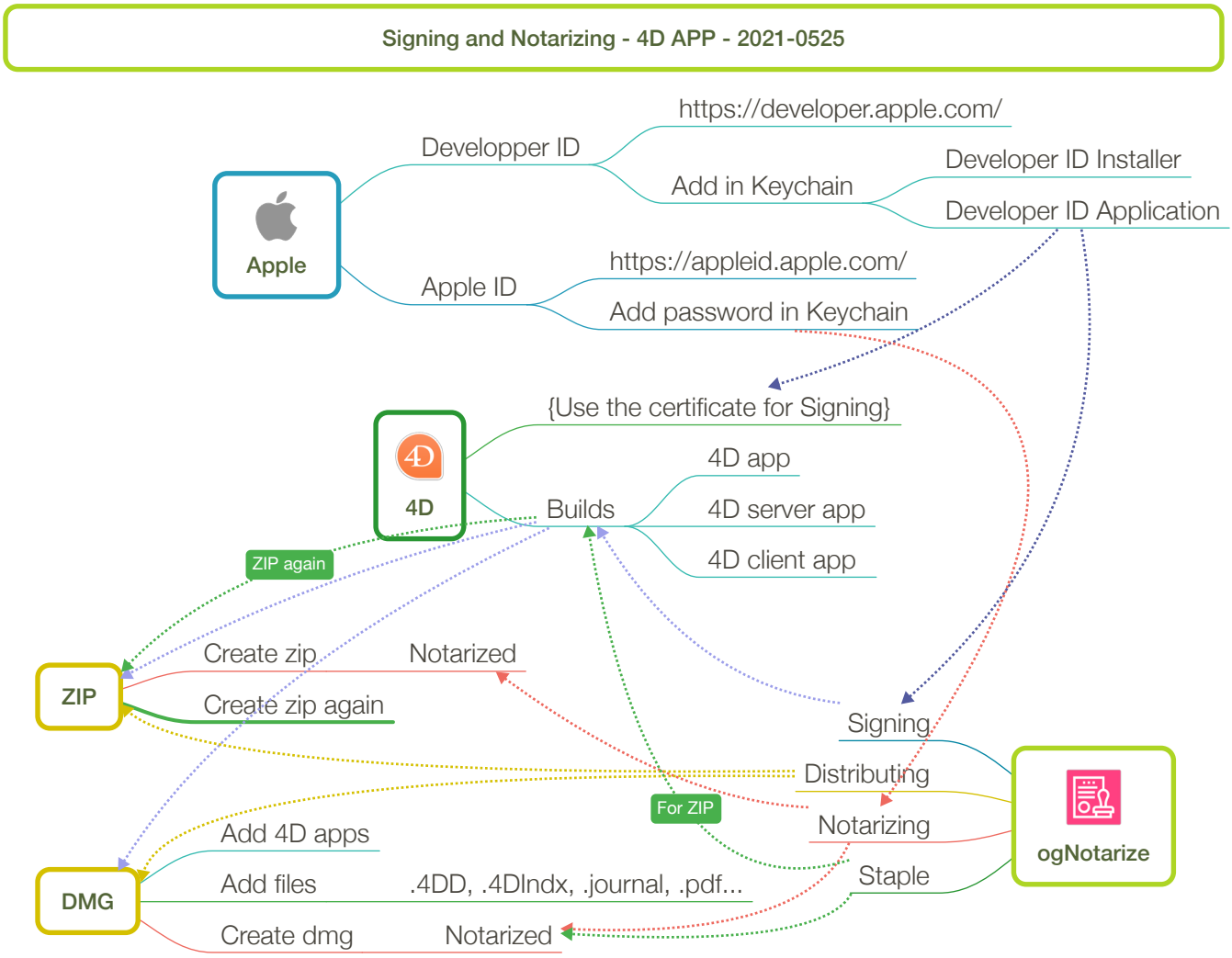
4D v19 integrate a Signing module that works quite well, but with missing features:

- if you launch the app
- if you modify anything inside the signed app
- if you want to add a folder, ie a web folder

the Signature is broken. So the need of a dedicated Signing process is still relevant for now. And the Notarizing process is not managed within the 4D build process. This step is difficult to manually do, too.

This is the purpose of this Oviyam module: store all the informations about your 4D products, paths, options, and do the full Signing and Notarizing with as less clicks as possible.

Thanks to Keisuke Miyako for its sample methods "4d-utility-build-application" at https://github.com/miyako/4d-utility-build-application, fully remastered and transformed into a component named "ogNotarize", and to 4D on how to Sign with the SignApp.sh file internally.



Signing

This process needs your 4D certificate to garantie that your software is from a well known developer.

Distributing

This step is to build a “distribution” that can be either a zip / dmg / pkg file, ready to distribute to your clients. All the software in it must be Signed in order to be Notarized next. Even for any web folder containing any executable files, like js...

Notarizing

The Notarizing process can be:

- altool, it sends your zip/dmg/pkg file to Apple, then it asks till an answer to know if your “distribution” was accepted or not (deprecated).
- notarytool, it’s an all in one process, it sends and wait for the answer in one command line.

Staple

You then staple your distribution with the Notarizing digital stamp. In case of a zip, you must staple the app again and do a new zip again - the zip itself can’t be staple.

About certificates

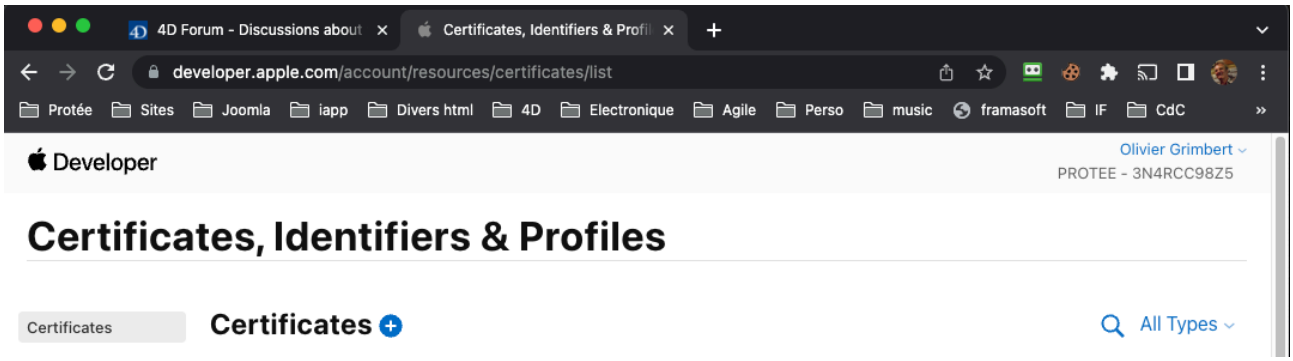
Before using Oviyam and its Notarizing module, you must install some certificates in the mac where you want to use it.

Beginning

Go to Apple, in your Team account https://developer.apple.com/ then click on “Account”. Enter your Apple ID and your password to login.

Then see on “Certificates, Identifiers & Profiles” and choose “Certificates”.

Click on “Certificate” ⊕ to add one.



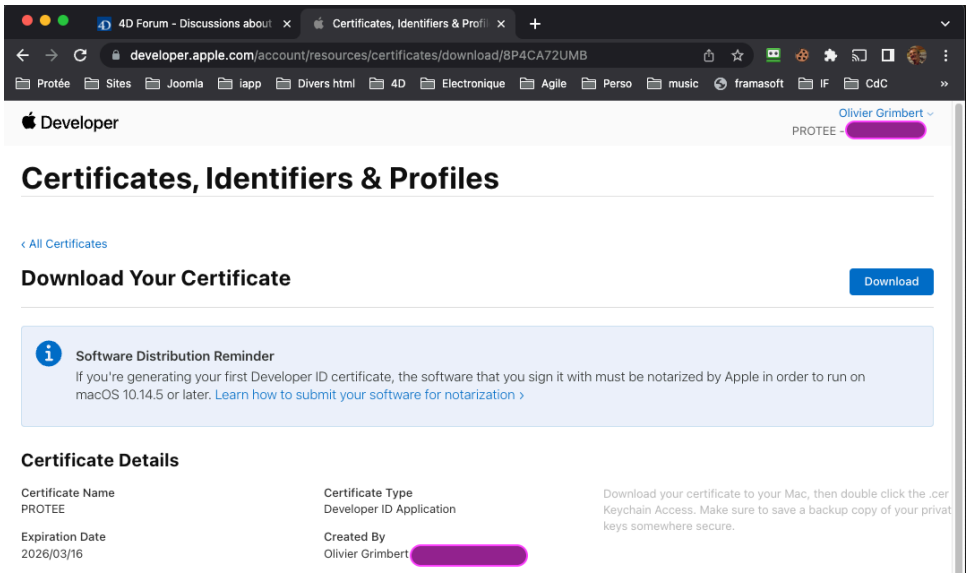
Developer ID Application *is a certificate used to sign the app before distribution outside the Mac App Store. A 4D app may be signed using this type of certificate for deployment.*

Developer ID Installer *is a certificate used to sign an installer containing an app sign using a “Developer ID Application” certificate. The installer is either a disk image (.dmg) or a package (.pkg). A simple zip archive is not considered to be a safe form of distribution, since its content can be altered during transport. A signed 4D app may be packaged and signed using this type of certificate for deployment over a network (AirDrop, HTTP, FTP, etc.). Copying an app from a connected external drive does not require an installer.*

Choose

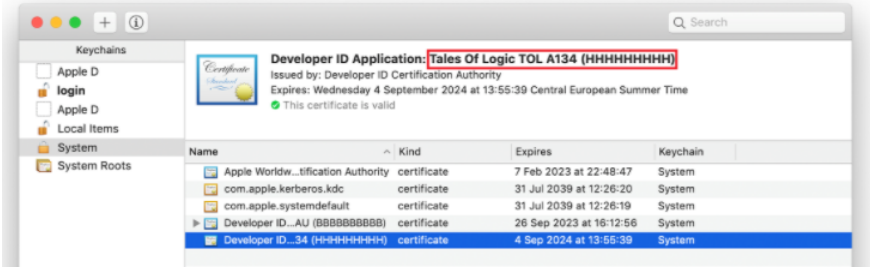
- “Developper ID Application” for regular 4D app, zip, dmg…
- “Developper ID Installer” for .pkg files

And click Continue. After that, your certificate is created and you need to click on “Download”, to download your certificate to your Mac, then double click the .cer file to install in Keychain Access.



Signing in 4D v19

- To build your 4D software with the Signing option
  - See here : https://developer.4d.com/docs/en/Project/building.html#os-x-signing-certificate
  - Licenses & Certificate page, fill in with your Apple certificate name



Ok! You got at first a signed App!

IMPORTANT: Once signed, you must NOT launch your 4D application before notarizing, or it will be rejected, as signing will be removed. But using Oviyam, you may do it as a new signing is done at the begin of the process.

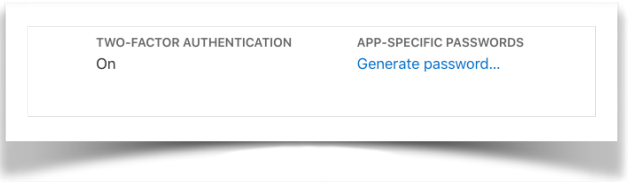
Notarizing

That is the last piece of the cake. https://blog.4d.com/how-to-notarize-your-merged-4d-application/

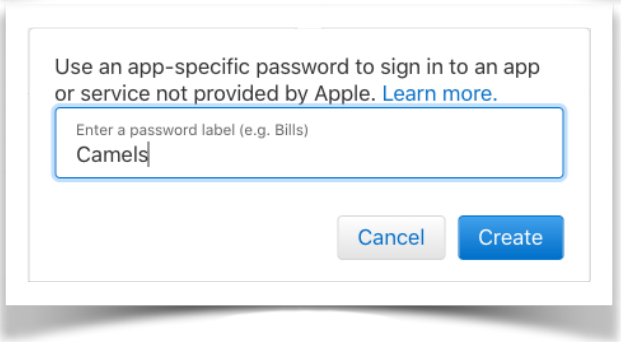
Before notarizing your first application, you'll need to generate an app-specific password for altool.

For signing, you have to

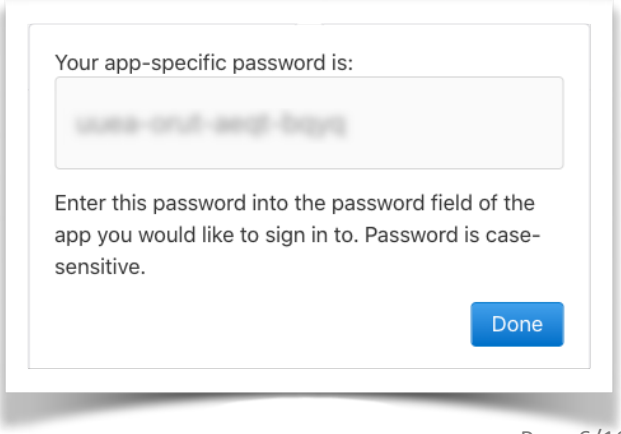
- go to your Apple account https://appleid.apple.com/ and login
- Check if the TWO-FACTOR AUTHENTICATION is on (mandatory for Notarizing)



- then in the "Sign-in and Security" section, click on "App-specific passwords". Add one if you need a new one.
- enter a password label (whatever you want) as requested and click the "Create" button.



- Then copy the password.



For altool

You can now store the password as a keychain item, using your Apple ID and the app-specific password:

- *security add-generic-password -a "apple-id" -w "app-specific-password" -s "altool"*

Then you can fill in Oviyam “password (app-specific)” with "@keychain:altool"…

Eventually, you’ll fall in the need to give the rights to altool. Explained here https://stackoverflow.com/questions/32976976/how-should-the-keychain-option-be-used-for-altool/56396125#56396125

For notarytool

- *xcrun notarytool store-credentials "notarytool" --apple-id "apple-id" --team-id "team-id" --password "app-specific-password"*

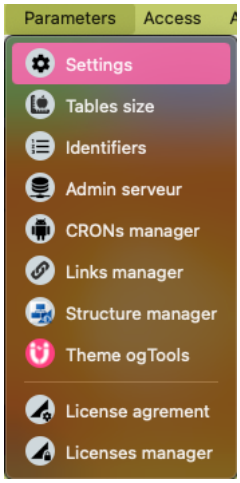
Then you can fill in Oviyam “keychainProfile” with “notarytool".

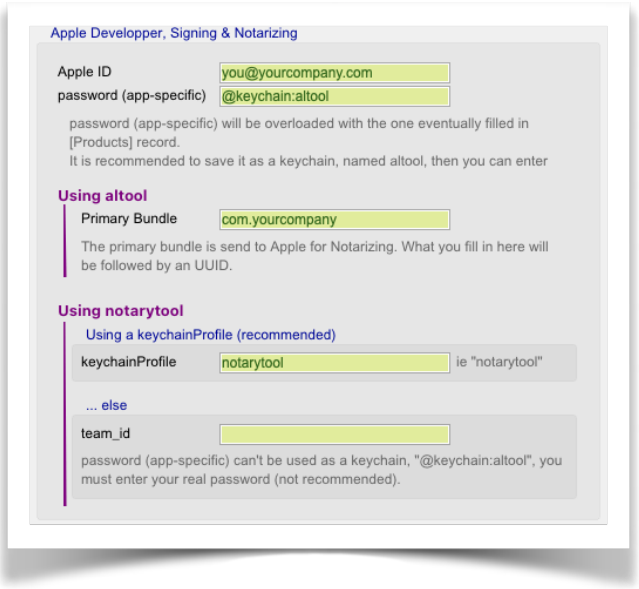
This is the simplest and more secure way to use your identifiants.

Oviyam configuration

Before using Oviyam and its Notarizing module, you must tell Oviyam about your certificates that will be used for the Notarizing process.

Goto “Parameters, Settings”, tab “APIs, Apple dev”.



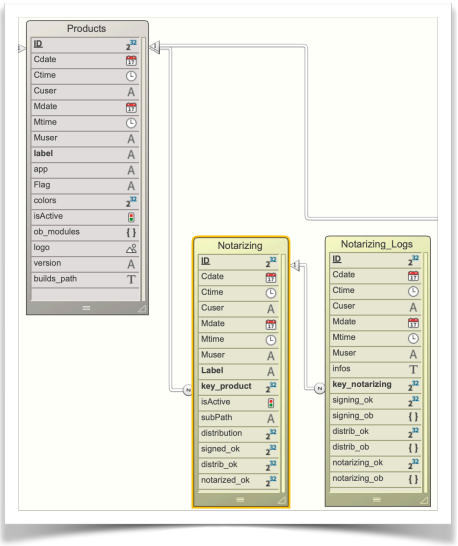


- **Apple ID** is your Apple ID, usually an email
- **app-specific password** if you followed the steps before, you may fill it with “@keychain:altool”: this will be replaced automatically by the password stored in your Keychain with the name “altool”.
- Old notarizing (altool)
  - **primary bundle**, like com.yourCompagny… this is used followed by an UUID when sending the file for Notarizing.
- New notarizing (notarytool)
  - **keychainProfile** if you followed the steps before, you may fill it with “notarytool”

If you want to manage more than one “app-specific password” - one per product, just place them in the [Products] records, when not empty, this will be added to **app-specific password** and **keychainProfile**. For “-oviyam”, it will seek for “@keychain:altool-oviyam” and “notarytool-oviyam”

Oviyam Notarizing

This module use the already existing table [Products] in Oviyam, and two new added tables [Notarizing] and [Notarizing_Logs].



[Products] table

This [Products] table is where, for all your different products, you give the root build's folder path for all your related builds to that products.

The root folder is where the distributions are made (zip, dmg, pkg…).

Usually, for a product, you'll have your 4D engined monoposte, and server, and client.

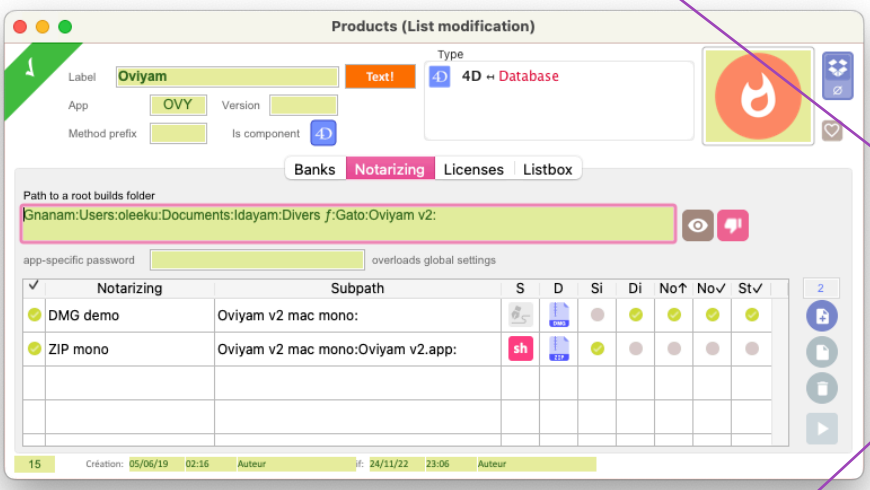
Each of them may be contained in a folder with some demo files:

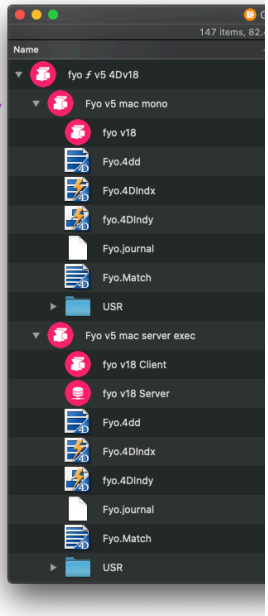
- like pdf, 4dd file, etc.

Here,

- root build's folder: …fyo ƒ v5 4Dv18:
- the app-specific password, left empty to use the global in parameters.

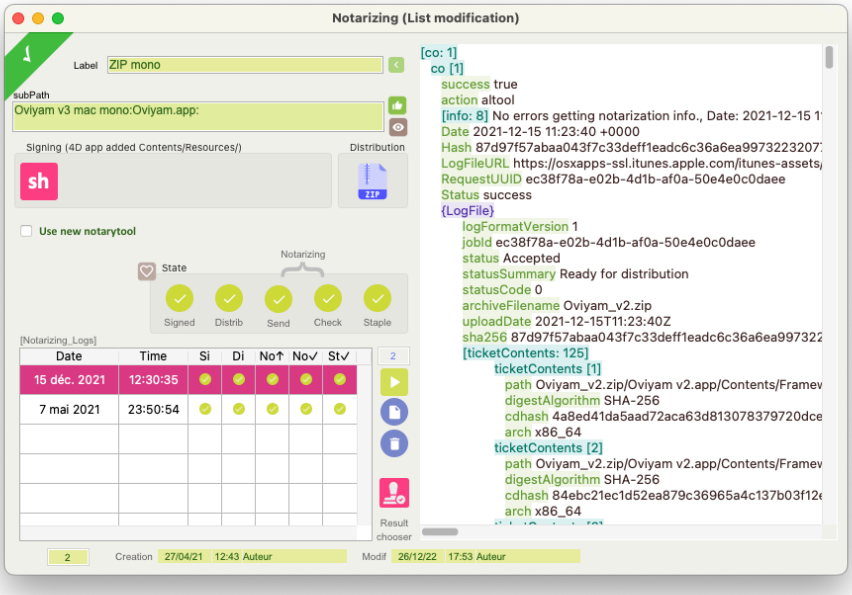
Then, for a specified product, you need to create





- as many [Notarizing] records as many you have builds (monoposte, server, client, in zip, dmg, pkg…, or demo folders):
  - 4D engined monoposte folder Fyo v5 mac mono
  - 4D engined server folder: Fyo v5 mac server exec
- and in it, you must put a subFolder (drag&drop) to an engine app, or even a folder (a full demo folder with .4DD file and so one).

And here the [Notarizing] record:

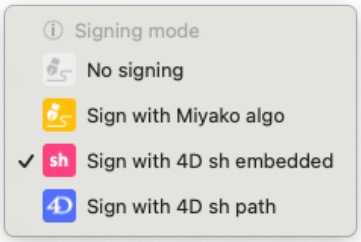


You give a label to that Record, and you select or drag' drop a file on subPath to define the subPath to the .app package, related to the build's root folder defined in [Products] — if a subPath, only the subPath is added here.

Don't forget to choose the distribution you want, as zip, dmg, pkg file type.

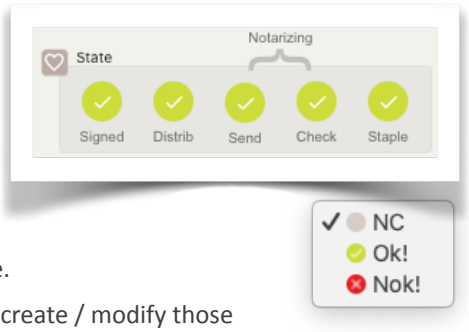
The options:

- Choose how to sign:
  - No signing, supposed to be already made
  - Sign with Miyako algo (in "4d-utility-build-application")
  - Sign with 4D sh embedded (at this time, from last v19 R6 copied into the component ogNotarize).
  - Sign with 4D sh path: give a path to a 4D.app



- "Do Signing" and "Clear Signing before" are option related to Signing, when it is relevant. If your 4D engined app is already signed with your 4D build, you can uncheck "Do Signing".
- "Clear Signing before" checked will ask the signing process to before all, unsign the entire app.
- A checkbox "Use new notarytool" will use it rather than the (deprecated) altool.

The "state" Group box contains the 5 states of the five possible steps int he process for Notarizing, and are automatically updated with the result of each action (see below).



However, you have a "favorite" button on the right top to check in real the file for the Signed and Notarized state.

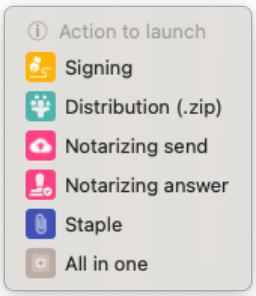
You can also change manually the state of every process state button, just in case you need to manually update those indicators.

There is a listbox for the [Notarizing_Logs] which stores every action made.

You can modify / delete these records, but usually you won't have to! You create / modify those record with the "Play" button.

If a line is selected, the result of the action is stored in the selected record, else a new record is created:

- [Notarizing_Logs] not selected, a full new record is added.
- [Notarizing_Logs] selected, selected record is modified.



Choose an action to execute, or choose “All in one” to get all being executed.

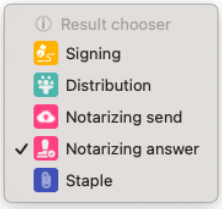
- Execute the Signing process
- Execute the Distribution process
- Execute the Notarizing send process
- Execute the Notarizing answer process
- Execute the staple process
- All in one: Execute all the process in a chain.

When the subPath is not an app, the “Signing” action is dimmed.

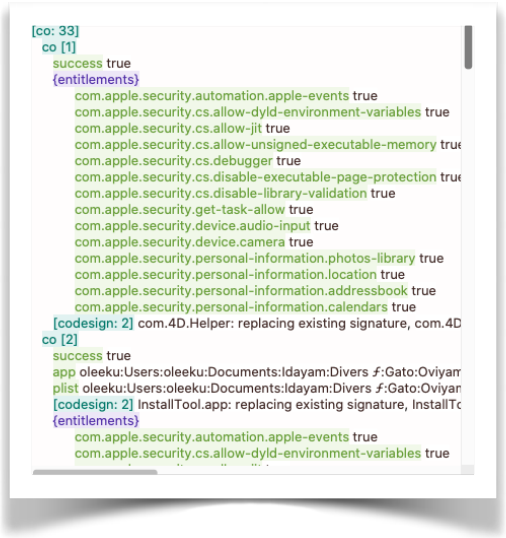
In mode “Use new notarytool”, the answer action is made during the same send action, so this action is dimmed.

After each action, the resulting “Collection” result is stored in the [Notarizing_Logs].

It is displayed on the right of the record, to allow you to understand any issues and errors you might get.



You choose which one is displayed with the button and menu “Result chooser”.

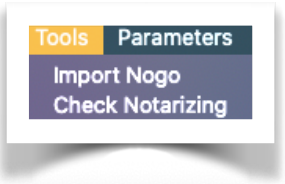


You may get a fast internet connection for Notarizing, as usually a 4D engine app is easily more than 300 Mb.

A zip file can’t be staple. So when you ask for the staple action, and it is a zip on an app, Oviyam will automatically staple the app file itself and then remake a zip file at the same place. So the app inside your zip is staple.

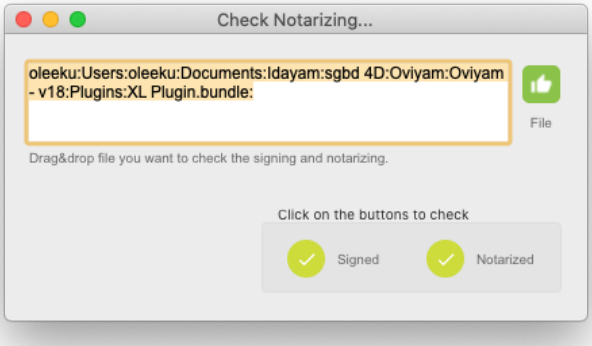
A .4dbase folder (package) can’t be staple too. In that case, for components, you need to use .dmg file.

Tool to Check Notarizing



Menu “Tools/Check Notarizing”

Just select a file, or drag & drop it to the path text zone. The Signed and Notarized indicators are calculated.



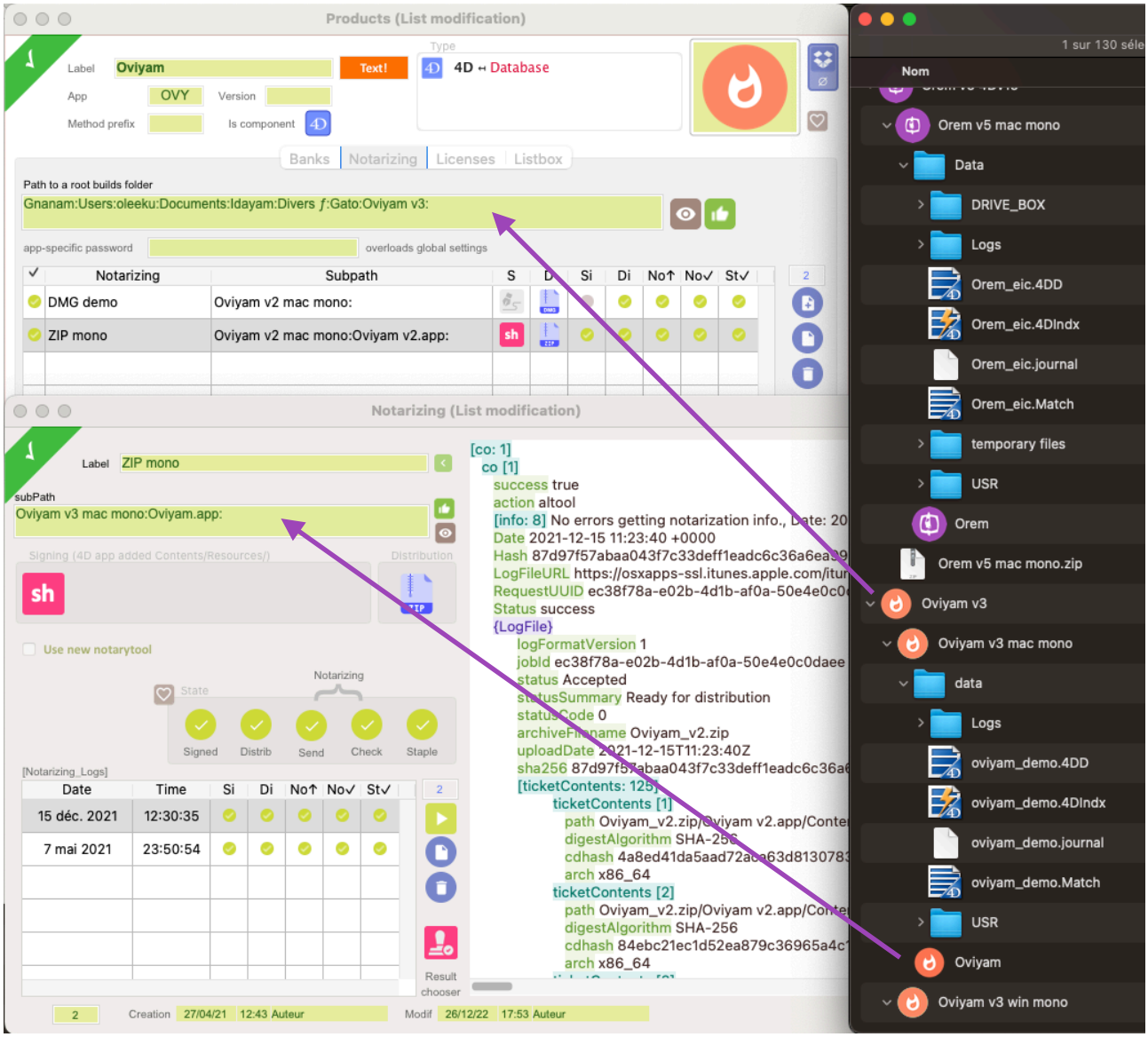
You can ask for a re-calculation by clicking on the indicator(s).

In practical

You have to indicate to Oviyam:

- the build's root folder in [Products]. This is where distributions (zip, dmg, pkg) are made.
- and for each distribution (mono, server, client, dmg demo…), in [Notarizing], each subPath

You can do this by Drag and Drop.



Select the option's checkboxes, normally automatically properly checked when you choose your subpath.

Then choose the action below in the menu:

All in one: Execute all the process in a chain. And wait till the magic…

Appendices

Signing

How to know if an app is well signed?

https://macissues.com/2015/11/06/how-to-verify-app-signatures-in-os-x/

Check for signing

- codesign --verify --verbose /Applications/AppName.app

More verbose:

- spctl --assess --verbose /Applications/Safari.app

You get a method for code sign check in method “xbuild_codesign_check”

Notarizing

That is the last piece of the cake.

Before notarizing your first application, you’ll need to generate an app-specific password for altool.

For signing, you have to go to your Apple account https://appleid.apple.com/ and In the Security section

- Check if the TWO-FACTOR AUTHENTICATION is on (mandatory for Notarizing)
- click the “Generate Password” option below the “App-Specific Passwords” option, enter a password label as requested and click the “Create” button.

You can now store the password as a keychain item, using your Apple ID and the app-specific password:

- security add-generic-password -a " <apple_id> " -w " <password> " -s "<altool>"

Take care of the right " to be used…

Eventually, you need to give the rights to altool. Explained here https://stackoverflow.com/questions/32976976/how-should-the-keychain-option-be-used-for-altool/56396125#56396125

CREATING A ZIP ARCHIVE OR A DMG IMAGE

Because .app bundles cannot be directly uploaded to the notary service, you’ll need to create a compressed archive containing your signed application:

- /usr/bin/ditto -c -k --keepParent "<path_to_app>" "<path_to_zip_archive>"

Note that you could also upload a .dmg image instead of a .zip archive.

UPLOADING YOUR APPLICATION

You can now upload your application for notarization using the following command:

- xcrun altool --notarize-app --primary-bundle-id "<primary_bundle_identifier>" --username "<apple_id>" --password "@keychain:altool" --file path_to_zip_archive

Note: you can use any name as a primary bundle identifier, but it must not contain spaces, underscores or anything other than letters, numbers, hyphens, and periods.

If the upload succeeds, a request UID is returned. Save this to use later when checking the status of your notarization request.

CHECKING THE NOTARIZATION STATUS

The notarization process may take up to an hour. When it's finished, you'll receive an email indicating the outcome. Since Apple has eased notarization prerequisites until January 2020, it's advised that you also request a detailed report to verify if it contains any warnings that may prevent notarization after January 2020.

A detailed report can be requested with the following command, which will return a status and a log file URL:

- xcrun altool --notarization-info <request_uid> -u "<apple_id>" -p "@keychain:altool"

STAPLING THE TICKET TO YOUR APPLICATION

In order to make sure that Gatekeeper knows that your application is notarized (even when no network connection is available at first launch of the application), it's recommended that you attach the ticket produced by notarization to your application. This process is called "stapling".

If you used a .zip archive for notarization, please note that the stapler tool must be run against the .app bundle originally added to the zip archive because you can't staple a .zip archive. If you used a .dmg image, you can directly staple your .dmg image.

Stapling is processed with the following command:

- xcrun stapler staple <your_app_or_dmg_file>

Once all is said and done, you can then create a new .zip archive containing the stapled application for distribution.

UPDATE

Recent macOS versions (BigSur/xcode 12) might require changes.

Apple now recommends to register your password using:

- xcrun altool --store-password-in-keychain-item "altool" -u "<apple_id>" -p "password"

And then not to specify the user name (apple_id) any longer using -u or –username when uploading or checking the application. For latest updates for xcrun altool open your Terminal and enter:

- xcrun altool —help

Research

Apple: https://developer.apple.com/documentation/xcode/notarizing_macos_software_before_distribution

Notarization gives users more confidence that the Developer ID-signed software you distribute has been checked by Apple for malicious components. Notarization is not App Review. The Apple notary service is an automated system that scans your software for malicious content, checks for code-signing issues, and returns the results to you quickly. If there are no issues, the notary service generates a ticket for you to staple to your software; the notary service also publishes that ticket online where Gatekeeper can find it.

When the user first installs or runs your software, the presence of a ticket (either online or attached to the executable) tells Gatekeeper that Apple notarized the software. Gatekeeper then places descriptive information in the initial launch dialog to help the user make an informed choice about whether to launch the app.

- First signing, Second notarizing

Forum

https://discuss.4d.com/t/v17r5-mac-notarization-using-miyakos-code/14396/4

https://discuss.4d.com/t/notarization-errors/11149/7

Gatekeeper

https://support.apple.com/en-us/HT202491

Apple

Signing your app

https://developer.apple.com/developer-id/

Get it notarized

https://developer.apple.com/documentation/xcode/notarizing_macos_software_before_distribution?preferredLanguage=occ

Grant the appropriate entitlements

Hardened Runtime Entitlements

- Disable Library Validation Entitlement: Normally, an app built using Xcode has the com.apple.security.get-task-allow entitlement during development, to facilitate debugging by circumventing certain security checks. Xcode automatically removes this entitlement for deployment during the export phase. This is already done for the 4D app itself. Plugins, on the other hand, might need the com.apple.security.get-task-allow entitlement in order to be debugged in the context of a host executable. To allow this, you need to enable the com.apple.security.cs.disable-library-validation entitlement (you enable it to disable the protection).
- Location Entitlement: There are no native 4D commands to access location services, but you may need this if you use plugins that do.
- Photos Library Entitlement: There are no native 4D commands to access photos, but you may need this if you use plugins that do.
- Audio Input Entitlement: There are no native 4D commands to record audio, but you may need this if you use plugins that do.

- Camera Entitlement: There are no native 4D commands to record video, but you may need this if you use plugins that do.
- Address Book Entitlement: There are no native 4D commands to access contacts, but you may need this if you use plugins that do.
- Calendars Entitlement: There are no native 4D commands to access calendars, but you may need this if you use plugins that do.
- Apple Events Entitlement: If the app runs AppleScript via osascript, this entitlement should not be necessary, but you may need this if ScriptingBridge or NSAppleScript is used by a plugin.

Find an appropriate certificate

5 different certificates types to choose from:

- Mac Development (Mac Developer) - testing
- ~~Mac App Distribution (3rd Party Mac Developer Application)~~
- ~~Mac Installer Distribution (3rd Party Mac Developer Installer)~~
- Developer ID Application
- Developer ID Installer

Developer ID Application is a certificate used to sign the app before distribution outside the Mac App Store. A 4D app may be signed using this type of certificate for deployment.

Developer ID Installer is a certificate used to sign an installer containing an app sign using a “Developer ID Application” certificate. The installer is either a disk image (.dmg) or a package (.pkg). A simple zip archive is not considered to be a safe form of distribution, since its content can be altered during transport. A signed 4D app may be packaged and signed using this type of certificate for deployment over a network (AirDrop, HTTP, FTP, etc.). Copying an app from a connected external drive does not require an installer.

Reading

Miyako component and documentation

https://github.com/miyako/4d-utility-build-application

https://miyako.github.io/2019/06/17/notarization.html

https://miyako.github.io/2020/02/07/notarization-after-february.html

On 4D

4D Blog https://blog.4d.com/how-to-notarize-your-merged-4d-application/

4D doc v18 https://developer.4d.com/docs/en/Project/building.html

And a Submit 2020 video https://events.4d.com/summit2020/session/4d-application-notarization-2/

4d-utility-build-application-master

Initialy from Miyako. This component consists in two part, one for Signing, one for Notarizing.

In 4D v18, 4D's built-in signing script has been updated to meet all of Apple's requirements for notarization. 4D now performs a recursive signature of all of the package contents and a secure timestamp is included with the signature. Hardened runtime is enabled and an entitlements file is provided. In order to avoid any feature restriction, all capabilities are set to "true". So this part of the component is not helping anymore.

But the Notarizing part is accurate.

You can use this component for :

- Create your file for notarizing
  - hdiutil or ditto, to get a .dmg or a .zip file.
  - pkgbuild to get a .pkg - need a special apple id for that
- Set your Xcode path
- altool to send an check the result
- staple your file (other then .zip)